



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Proceso de Reclutamiento Agrupación: UNAM Motorsports —
División de Data Acquisition

Actividad de Reclutamiento: Diagnóstico de Telemetría y Detección
de Anomalías (Fórmula SAE)

Reporte Técnico Final: Auditoría, Saneamiento y Diagnóstico de un Registro de Telemetría Vehicular

Autor:

- Francisco José Coutiño Morales (Núm. Cta: 425121623)

16 de julio de 2026

Índice

1. Introducción	2
1.1. Flujo de Trabajo y Metodología de Desarrollo	2
2. Arquitectura del Pipeline de Datos y Lógica de Diagnóstico	2
2.1. Auditoría y Saneamiento del Dataset	2
2.2. Validación Estadística: la Autocorrelación como Prueba de Integridad	3
3. Acondicionamiento de Señal: Filtrado y Estructuras Algorítmicas	4
3.1. Filtrado Digital: Butterworth de Fase Cero	4
3.2. Detección de Intervalos: Algoritmo Genérico de Persistencia	4
3.3. Cálculo Vectorial: Magnitud de Fuerza G y Diagrama G-G	4
4. Diseño del Pipeline: Módulos y Responsabilidades	4
4.1. <code>parse_value()</code> — Saneamiento de Origen	5
4.2. <code>butter_lowpass_filtfilt()</code> — Acondicionamiento	5
4.3. <code>find_runs()</code> — Motor de Detección de Fallas	5
4.4. <code>dashboard.py</code> — Visualización Interactiva (Streamlit)	5
5. Detección y Corrección de Errores en el Dataset (“Bugs” de Instrumentación)	5
6. Resultados del Análisis: Diagrama G-G y Eventos Confirmados	6
7. Conclusiones	6

1. Introducción

El presente documento detalla el proceso de auditoría, saneamiento, acondicionamiento de señal y detección de anomalías aplicado sobre el registro de telemetría (`data.csv`, muestreado a 100 Hz) proporcionado para la actividad de reclutamiento de la división de Data Acquisition de UNAM Motorsports. El objetivo del ejercicio, tal como lo establece la actividad, es doble: por un lado, construir un pipeline determinista capaz de acondicionar señales inerciales, calcular la magnitud de fuerza G y generar el Diagrama G-G bajo la normativa Fórmula SAE; por otro, actuar como “última línea de defensa de la calidad del dato”, detectando y documentando cualquier falla de instrumentación antes de que el registro se use para decisiones de ingeniería reales (correlación de set-up, validación de simulaciones, etc.).

1.1. Flujo de Trabajo y Metodología de Desarrollo

El desarrollo siguió un enfoque exploratorio e iterativo, dividido en cuatro fases:

1. **Fase de Auditoría de Origen:** Al intentar cargar `data.csv` con `pandas.read_csv`, las columnas numéricas no parseaban como flotantes válidos. En vez de asumir un error trivial de formato y aplicar una corrección arbitraria, se realizó una auditoría forense del archivo: inspección de patrones de dígitos, pruebas de hipótesis de escala, y verificación cruzada directamente contra la celda fuente en Google Sheets (barra de fórmulas), confirmando que el valor almacenado era un entero puro sin punto decimal.
2. **Fase de Saneamiento y Validación Estadística:** Una vez corregida la escala, se aplicó una prueba de autocorrelación (lag-1) sobre las 30,000 muestras para validar que el registro corregido fuese físicamente consistente con una señal continua de vehículo a 100 Hz — no basta con que los números “se vean bien”, deben comportarse como telemetría real.
3. **Fase de Acondicionamiento y Análisis:** Se diseñó el filtro pasa-bajas de fase cero, se calculó la magnitud de fuerza G sobre la señal filtrada, y se generó el Diagrama G-G con su diagnóstico de asimetría.
4. **Fase de Detección de Fallas y Visualización:** Se implementaron algoritmos genéricos de detección de corridas sostenidas (>100 ms) para fallas de sensor y violaciones de plausibilidad, documentados en un notebook reproducible y expuestos en un dashboard interactivo (Streamlit) para inspección visual del registro completo.

2. Arquitectura del Pipeline de Datos y Lógica de Diagnóstico

2.1. Auditoría y Saneamiento del Dataset

El hallazgo más relevante de todo el ejercicio ocurrió antes de escribir una sola línea de análisis de señales: el archivo fuente no era parseable como telemetría estándar. Valores

como `speed_kmph = 36.075.011.221.318.400` no correspondían a ningún formato numérico válido en Python.

Justificación del diagnóstico: en lugar de descartar el archivo o aplicar una corrección heurística sin evidencia, se contrastó el valor crudo directamente contra la fuente original en Google Sheets, confirmando que la celda almacena el entero 36075011221318400 sin punto decimal alguno. Los puntos visibles en el CSV exportado corresponden únicamente al separador de miles aplicado por el *formato de celda* (visual), no al contenido real del dato. Esto descarta un error de codificación de caracteres y apunta a un **bug sistemático de escala en la etapa de adquisición**: cada lectura fue multiplicada por $1e15$ (presumiblemente para preservarla como entero de precisión completa) antes de almacenarse, y el archivo se exportó reflejando el formato visual de la celda en lugar del valor matemático.

La corrección aplicada, documentada íntegramente en `parse_value()`, es:

```
valor_real = int(dígitos_sin_puntos) / 1e15
```

Durante la auditoría se identificaron, además, dos anomalías de formato adicionales que exigieron tratamiento diferenciado:

- **Notación científica corrupta (formato europeo, coma decimal):** 6 muestras aisladas (p. ej. `-3,41E+10`) no seguían ni el patrón de entero escalado ni el de flotante estándar. Se clasificaron como lectura corrupta puntual y se interpolaron, ya que no cumplen la regla de persistencia de 100 ms (Sección 3.2) para ser declaradas falla.
- **Valores cortos literales sin escalar** (p. ej. `accel_x = 1.0` o `0.0`): no siguen el patrón de entero gigante, por lo que `parse_value()` los interpreta como el valor real directo. Este comportamiento bimodal del propio archivo terminó siendo la pista que condujo a los hallazgos de saturación y caída de señal reportados en la Sección 5.

2.2. Validación Estadística: la Autocorrelación como Prueba de Integridad

Corregir el formato no es suficiente para garantizar que un dato sea utilizable: un número bien formado puede seguir siendo físicamente imposible. Se calculó la autocorrelación de retraso 1 (`lag-1`) sobre las 30,000 muestras corregidas de cada canal. Para telemetría real a 100 Hz, este valor debe ser cercano a +1 (un vehículo tiene inercia; su velocidad y aceleración no pueden ser estadísticamente independientes entre una muestra y la siguiente, 10 ms después). El resultado obtenido fue de **-0.011 para `speed_kmph`** y **+0.005 para `accel_x`**, es decir, esencialmente ruido independiente. Este hallazgo se documenta como limitación explícita del análisis: cualquier interpretación física de los resultados posteriores (asimetría del Diagrama G-G, magnitud de carga) debe leerse como demostración metodológica del pipeline y no como caracterización real del comportamiento dinámico de un vehículo.

3. Acondicionamiento de Señal: Filtrado y Estructuras Algorítmicas

3.1. Filtrado Digital: Butterworth de Fase Cero

Justificación del filtro: se seleccionó un filtro Butterworth de orden 4 con frecuencia de corte de 15 Hz, aplicado mediante `scipy.signal.filtfilt`. La elección del algoritmo `filtfilt` (en lugar de `lfilter`) no es arbitraria: aplica el filtro dos veces (adelante y atrás en el tiempo), cancelando el desfase de fase que introduciría un filtro IIR convencional, cumpliendo el requisito explícito de la actividad de no introducir retraso temporal. La frecuencia de corte se justificó mediante inspección de la densidad espectral de potencia (PSD) de `accel_x`; si bien el espectro resultó casi plano — otro síntoma de la anomalía de la Sección 2.2 —, se conservó el criterio estándar de la industria motorsport (resonancia de chasis/suspensión típicamente por encima de 20-30 Hz) para justificar el corte elegido.

Por qué el valor crudo es engañoso: sin filtrar, un único pico de ruido de alta frecuencia puede dominar el máximo reportado de $-G$, llevando a decisiones de ingeniería erróneas (p. ej. sobredimensionar un componente estructural en respuesta a un pico de ruido inexistente físicamente). Por ello, $G_{mag} = \sqrt{accel_x_filt^2 + accel_y_filt^2}$ se calcula exclusivamente sobre la señal ya condicionada.

3.2. Detección de Intervalos: Algoritmo Genérico de Persistencia

Tanto para fallas del sensor inercial como para violaciones de plausibilidad, la actividad exige ignorar falsos positivos de una sola muestra: una condición solo se declara falla si se sostiene de forma continua por más de 100 ms (>10 muestras a 100 Hz). En lugar de escribir lógica ad-hoc distinta para cada caso, se implementó una única función reutilizable, `find_runs()`, que recorre la serie booleana de la condición evaluada y agrupa corridas contiguas, descartando aquellas por debajo del umbral de persistencia. Esta decisión de diseño (una sola función genérica en vez de dos implementaciones paralelas) redujo la superficie de error y permitió aplicar exactamente el mismo criterio tanto a la caída de señal del IMU como a la violación de plausibilidad freno-acelerador.

3.3. Cálculo Vectorial: Magnitud de Fuerza G y Diagrama G-G

El Diagrama G-G se construye graficando `accel_y_filt` contra `accel_x_filt`; representa la envolvente de fricción máxima que el vehículo explota en cada eje. Su lectura no es meramente gráfica: la forma y simetría de la nube de puntos es, en sí misma, un diagnóstico de instrumentación.

4. Diseño del Pipeline: Módulos y Responsabilidades

El sistema se organizó siguiendo el Principio de Responsabilidad Única, separando claramente saneamiento, acondicionamiento, detección y visualización:

4.1. `parse_value()` — Saneamiento de Origen

Corrige la codificación de escala y descarta muestras corruptas en notación científica, devolviendo NaN para su posterior interpolación (Sección 2.1).

4.2. `butter_lowpass_filtfilt()` — Acondicionamiento

Encapsula el diseño y aplicación del filtro Butterworth de fase cero, parametrizado por frecuencia de corte y orden, reutilizable para cualquier canal inercial.

4.3. `find_runs()` — Motor de Detección de Fallas

Función genérica de detección de corridas sostenidas, reutilizada tanto para la caída de señal del sensor (`accel_x == 0.0`) como para la violación de plausibilidad (`brake_pressure > umbral AND throttle > umbral`), garantizando consistencia de criterio entre ambos tipos de falla.

4.4. `dashboard.py` — Visualización Interactiva (Streamlit)

Reutiliza íntegramente la misma lógica de saneamiento y detección del notebook (sin duplicar código de negocio), exponiendo un cursor temporal, gráficas de canales inerciales y de pedales, el Diagrama G-G con la posición actual resaltada, y un indicador visual crítico que cambia a estado de alerta en cuanto el cursor temporal entra en un intervalo de falla confirmada.

5. Detección y Corrección de Errores en el Dataset (“Bugs” de Instrumentación)

Durante el ciclo de auditoría se detectaron anomalías que no impedían la carga del archivo, pero comprometían la validez de cualquier análisis posterior. Fueron documentadas y tratadas de la siguiente manera:

1. **Vulnerabilidad de “Escala Fantasma”:** el pipeline de adquisición almacenaba cada lectura multiplicada por $1e15$ sin que el CSV exportado reflejara esa transformación, generando valores sin sentido físico aparente.
Solución implementada: se validó la hipótesis contra la fuente original (Google Sheets) antes de aplicar cualquier corrección, evitando “arreglar” el archivo con una suposición no verificada. Se corrigió únicamente tras confirmar 3 valores de referencia.
2. **Falsos Positivos de Saturación:** el valor exacto `accel_x = 1.0` aparece en el 42.5 % del registro (12,756 muestras), lo que a primera vista sugeriría saturación crónica del sensor. Tras aplicar la regla de persistencia de 100 ms, **ninguna corrida individual la supera** (la más larga es de exactamente 100 ms). Se descartó como falla confirmada y se reclasificó como ruido/artefacto intermitente — el caso de uso exacto para el que existe la regla de persistencia.

3. **Caída de Señal Confirmada:** se detectó una corrida sostenida de `accel_x` = 0.0 de 145.97 s a 147.46 s (150 muestras, 1.50 s), mientras `accel_y` continuó variando con normalidad en el mismo intervalo, aislando la falla al canal `accel_x` (p. ej. cable o conector de ese eje específico) y descartando una pérdida total de alimentación del IMU.

6. Resultados del Análisis: Diagrama G-G y Eventos Confirmados

El Diagrama G-G generado muestra una asimetría direccional marcada: la envolvente del canal lateral (`accel_y`) es varias veces más amplia que la del canal longitudinal (`accel_x`). En un registro real de pista, esta clase de asimetría se explicaría típicamente por desalineación del sensor respecto a los ejes del vehículo, diferencia de ganancia de calibración entre canales, sesgo del trazado (predominancia de curvas en una dirección), o saturación de uno de los canales. Dado el hallazgo de la Sección 2.2 (ausencia de estructura temporal), la explicación más probable en este registro específico es una diferencia de escala/generación entre canales, no una causa física de puesta a punto.

Respecto a la violación de plausibilidad (Regla T.4.2 — freno y acelerador simultáneos), de aproximadamente 10,700 muestras puntuales candidatas, **solo un evento sobrevive el filtro de persistencia: de 60.29 s a 60.40 s** (110 ms), con ambos pedales sostenidos consistentemente por encima del umbral de accionamiento durante todo el intervalo.

7. Conclusiones

El presente ejercicio de diagnóstico de telemetría deja tres aprendizajes centrales. Primero, que la calidad del dato no puede verificarse únicamente por su formato: un archivo que carga sin errores puede seguir siendo físicamente inválido, como evidenció la autocorrelación lag-1 prácticamente nula pese a la corrección exitosa de escala. Segundo, que la regla de persistencia de 100 ms no es un requisito cosmético: sin ella, el 42.5 % del registro habría sido reportado erróneamente como saturación crónica, y el 36 % como violaciones de plausibilidad, cuando en realidad solo un evento de cada tipo resultó ser una falla real sostenida. Tercero, que la trazabilidad de cada corrección —validar la hipótesis de escala contra la fuente original antes de aplicarla— es la diferencia entre un pipeline confiable y uno que enmascara errores bajo una corrección plausible pero no verificada.

Sobre el registro corregido se aplicó un filtro Butterworth de orden 4 (corte 15 Hz, fase cero) y se calculó la magnitud de fuerza G sobre la señal filtrada, evitando la sobreestimación que produciría el ruido crudo. El Diagrama G-G evidenció una asimetría direccional atribuible, en este caso, a diferencia de escala entre canales antes que a una causa física de set-up. Se confirmaron dos eventos: una caída de señal sostenida en `accel_x` (145.97 s–147.46 s) y una violación de plausibilidad freno-acelerador (60.29 s–60.40 s). **Recomendación:** no utilizar este registro para correlación de set-up ni validación de simulaciones sin auditar previamente la cadena de adquisición completa, dado que la falta de continuidad temporal apunta a un problema sistémico anterior al archivo entregado, no a ruido de sensor ordinario.

Bibliografía

- [1] NumPy Developers. (s.f.). *NumPy documentation*. NumPy. Recuperado de <https://numpy.org/doc/stable/>
- [2] The pandas development team. (s.f.). *pandas documentation*. pandas. Recuperado de <https://pandas.pydata.org/docs/>
- [3] The SciPy community. (s.f.). *scipy.signal.filtfilt — SciPy v1.x Manual*. SciPy. Recuperado de <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.filtfilt.html>
- [4] Streamlit Inc. (s.f.). *Streamlit documentation*. Streamlit. Recuperado de <https://docs.streamlit.io/>
- [5] SAE International. (2026). *Formula SAE Rules*. SAE International. Recuperado de <https://www.fsaonline.com/cdsweb/gen/DocumentResources.aspx>